# Mejoras recomendadas al generador de microservicios Quarkus

Proyecto agentIA_v2 | Rama feat/skill-api-generator-pstack-repair | Octubre 2026

## Resumen

El proyecto genera un servicio Quarkus por blueprint: usa el scaffolder oficial (quarkus-maven-plugin), genera entity, repository y resource con un LLM, verifica con Maven en Docker y repara con parches acotados. La arquitectura es sólida, pero casi todo el código Quarkus depende del LLM y los commits recientes son parches a sus errores (imports, persist, camelCase). La mejora de mayor impacto es pasar el CRUD a plantillas deterministas y dejar al LLM solo la lógica de negocio.

## Hallazgos principales

- Entity, Repository y Resource los escribe el LLM y luego se sanean con regex; es un patrón de síntoma recurrente.
- El perfil quarkus-3.yaml declara plantillas (templates/quarkus/*.jinja) que no existen en el repositorio.
- El contrato admite un solo serviceName por blueprint: no hay varios servicios, gateway ni mensajería generada, aunque la extensión de Kafka está en la allowlist.
- El ejemplo canónico del PUT hace persist(updated) sobre una entidad desacoplada, lo que falla o duplica; el modelo lo copia.
- La verificación depende de Docker y de mvn -o con el caché ~/.m2 precalentado.

## Plan de mejoras por prioridad

| # | Mejora | Esfuerzo | Impacto |
|---|---|---|---|
| 1 | Medir una línea base: correr demo_quarkus.json y registrar intentos de repair y archivos que fallan. | Bajo | Alto |
| 2 | Plantillas Jinja deterministas para Entity, Repository y Resource; el LLM solo agrega lógica de negocio. | Medio | Muy alto |
| 3 | Corregir el ejemplo canónico del PUT en profiles/quarkus-3.yaml. | Muy bajo | Alto |
| 4 | Generar los tests @QuarkusTest con RestAssured por plantilla (POST, GET, PUT, DELETE, 404). | Medio | Alto |
| 5 | Decidir el alcance de "microservicios": uno por blueprint o varios por blueprint. | Decisión | Estratégico |
| 6 | Smoke test end-to-end: mvn package, levantar el servicio y llamar a un endpoint. | Medio | Alto |

## 1. Medir antes de cambiar

Ejecutar el pipeline completo con blueprints/demo_quarkus.json y anotar cuántos intentos de repair necesita y qué archivos fallan. Asegurar antes que Docker y el caché de Maven estén listos para que los fallos reflejen la calidad del generador y no el entorno.

## 2. Plantillas deterministas para el CRUD

- Crear templates/quarkus/Entity.java.jinja, Repository.java.jinja y Resource.java.jinja (el perfil ya las declara) y renderizarlas desde el blueprint.
- Reservar el LLM para la lógica derivada de las user stories: métodos adicionales y validaciones.
- Resultado esperado: el CRUD básico compila siempre, el repair casi desaparece y la generación es rápida incluso con un modelo local de 7B.
- Elimina de raíz los parches recientes de imports, persist y camelCase.

## 3. Corregir el PUT canónico

Cambio de pocas líneas en profiles/quarkus-3.yaml. Versión recomendada:

```
@PUT
@Path("/{id}")
@Transactional
public Response update(@PathParam("id") Long id, Entity updated) {
    Entity existing = repository.findById(id);
    if (existing == null) {
        return Response.status(Response.Status.NOT_FOUND).build();
    }
    existing.setName(updated.getName());   // copiar campos editables
    return Response.ok(existing).build();  // el contexto transaccional hace el flush
}
```

## 4. Tests generados por plantilla

Un @QuarkusTest con RestAssured por entidad cubriendo POST, GET, PUT, DELETE y 404 es completamente predecible. Hoy es la parte más frágil del flujo y la que más consume repair.

## 5. Definir qué significa "microservicios"

- **Un servicio por blueprint (estado actual):** suficiente si cada ejecución produce un microservicio desplegable por separado. No requiere cambios.
- **Varios servicios por blueprint:** exige extender el contrato (lista de servicios, dependencias, mensajería) y generar un proyecto por servicio.
- Recomendación: empezar con la primera opción y tratar la segunda como una evolución posterior.

## 6. Verificación end-to-end

Añadir un smoke test que ejecute mvn package, levante el servicio y haga una petición a un endpoint. Es la única prueba de que lo generado es un servicio Quarkus funcional y no solo código que compila.

## Qué evitar

Seguir agregando reglas, regex al sanitizador y pitfalls al perfil. Cada commit reciente corrige un error del modelo y siempre aparecerá otro; las plantillas atacan la causa en lugar del síntoma.

## Orden de ejecución sugerido

- Primero: puntos 1, 3 y 2 (línea base, arreglo del PUT y plantillas de Entity/Repository/Resource).
- Después: punto 4 (tests por plantilla) y punto 6 (smoke test).
- Cuando haya decisión: punto 5 (alcance multi-servicio).